CLEAR-MoE++: Calibration-Driven Layer-Selective Expert Extraction from Pretrained Vision Transformers with Parallelism-Aware Routing and Runtime Co-Design

Md. Irtiza Hossain
Department of Computer Science and Engineering
Dhaka, Bangladesh
md.irtiza.hossain@g.bracu.ac.bd
Student ID: 20101481

***Abstract*—Modern image-recognition networks called *vision transformers* (ViTs) process every image patch with the same amount of computation regardless of whether the patch contains a background sky or a salient object boundary. This uniformity is wasteful: real images are semantically heterogeneous, yet two-thirds of all inference arithmetic is spent in the *feed-forward network* (FFN) sub-layers that execute identically for every input token. *Sparse Mixture-of-Experts* (MoE) architectures correct this by routing each token to only a small subset of specialised sub-networks ("experts"), but existing MoE vision models require training from scratch—an impractical barrier for practitioners with pretrained checkpoints. We present CLEAR-MoE++, a post-training pipeline that converts any frozen pretrained ViT into a sparse-compute MoE backbone in four phases: (1) score every FFN layer by its *activation multimodality* and *output sensitivity* using a small calibration set; (2) decompose only the highest-scoring late-stage layers into a shared low-rank basis (via truncated SVD) plus cluster-specific residual experts (via k-means), preserving common visual structure; (3) fit lightweight token routers; (4) deploy inference through seven router variants and six GPU dispatch backends. Three novel modules are contributed: ALFRouter—an auxiliary-loss-free router that achieves load-balanced expert assignment without a manually tuned penalty term; stream_fine—a fine-grained CUDA stream executor for overlapping expert computation; and DisaggregatedSim—a module that precisely measures the PCIe round-trip cost of disaggregated CPU-router + GPU-expert inference. On Imagenette (DeiT-Small, $E{=}4$ experts), CLEAR-MoE achieves 86.51% Top-1 accuracy (99.8% of the dense baseline 86.69%) with a 42.18% reduction in active parameters. SegFormer-B0 extraction on Cityscapes preserves the dense mIoU of 0.5757. cuBLAS batched-GEMM dispatch achieves 249,775 tokens/second—an 18.63$\times$ speedup over CPU-serial execution. DisaggregatedSim measures a PCIe round-trip penalty of 5.04 ms per batch (net transfer cost $\Delta{=}3.08$ ms above co-located serial), confirming that interconnect latency—not expert computation—bottlenecks disaggregated inference. Roofline analysis confirms that dispatch engineering, not expert GEMM optimisation, yields the greatest end-to-end latency return on bandwidth-limited hardware.**

***Index Terms*—mixture of experts, vision transformer, post-training extraction, conditional compute, sparse inference, semantic segmentation, dispatch strategies, roofline analysis, expert parallelism, PCIe overhead**

Code Availability: All source code and experiments are available at https://github.com/irtiza1999/Clear_Moe.

I. Introduction

A. Background: Vision Transformers and the Uniform-Compute Problem

A *vision transformer* (ViT) [1] divides an input image into a regular grid of non-overlapping $16{\times}16$-pixel patches, embeds each patch as a d-dimensional vector (a *token*), and processes all tokens jointly through L stacked *transformer blocks*. Each block contains two sub-layers: a *multi-head self-attention* (MHSA) layer that computes pairwise interactions between tokens, and a *feed-forward network* (FFN) that applies a two-layer MLP independently to each token. For a $224{\times}224$ image this produces $T = 196$ tokens; a standard 12-block DeiT-Small runs $196 \times 12 = 2{,}352$ FFN evaluations per image—all identical in structure regardless of whether the token is a featureless background region or an object edge.

The FFN accounts for approximately **two-thirds of all inference FLOPs** in standard ViT architectures [4], yet executes uniform computation for semantically heterogeneous tokens. This is the *uniform-compute problem*: the network devotes the same arithmetic to easy and hard tokens alike.

B. Sparse MoE as a Solution

Sparse Mixture-of-Experts (MoE) [3] replaces each uniform FFN with E specialised sub-networks (*experts*). A lightweight *router* (gating network) examines each token and selects only the top-k most relevant experts to process it, leaving the rest idle. The computational benefit is proportional to k/E: routing to 1 of 4 experts reduces active FFN compute by up to 75%. Critically, different experts can specialise for different visual concepts (textures, edges, object classes), improving the effective parameter utilisation.

Vision MoE models including V-MoE [4] and AdaMV-MoE [8] have demonstrated that expert routing can halve active inference FLOPs with negligible accuracy loss. However, these systems are trained from scratch—requiring enormous compute resources unavailable to practitioners with an existing pretrained checkpoint.

C. Post-Training Extraction: Gaps

Post-training expert extraction converts a pretrained dense FFN into experts without retraining [11]–[13]. Despite recent progress, three critical gaps remain:

1) **Layer selectivity**: prior methods expertise all FFN layers or a fixed fraction, ignoring the empirical finding that later transformer layers form more specialised representations [8] and thus benefit more from expert routing.
2) **Shared structure**: disjoint extraction assigns the entire FFN weight matrix to independent experts, discarding common visual structure (e.g., low-level edge detectors shared across all experts) and destabilising transfer to dense prediction tasks like semantic segmentation.
3) **Runtime faithfulness**: most extraction papers report reduced MAC (multiply-accumulate operation) counts—a theoretical measure—without demonstrating that structural sparsity translates into wall-clock latency reduction on real hardware.

D. CLEAR-MoE++: Contributions

CLEAR-MoE++ addresses all three gaps with the following contributions:

- **Calibration-driven layer scoring** $\mathcal{S}(l) = \alpha \cdot \text{sparsity}(l) + \beta \cdot \text{clusterability}(l) - \gamma \cdot \text{sensitivity}(l)$—selects only the most expertisable FFN blocks based on measured activation statistics.
- **Shared-basis plus residual expert decomposition** (truncated SVD + k-means)—preserves common visual structure in a shared weight component while allocating cluster-specific residuals to individual experts.
- **Seven router variants** spanning linear gates, MLP gates, expert-choice routing, communication-aware routing, path-consistent routing, self-routing, and the novel ALFRouter.
- **Six dispatch backends** (Naive, Grouped, Stream, Stream_fine, Triton, cuBLAS) benchmarked across four token-load imbalance scenarios to identify real-hardware winners.
- **Three novel runtime modules**: ALFRouter, stream_fine, and DisaggregatedSim with explicit PCIe overhead measurement.
- **24-cell ablation** and roofline analysis quantifying the compute vs. memory bottleneck of every pipeline component.

II. Background and Related Work

A. Key Terminology

Table I defines the domain-specific terms used throughout this paper.

TABLE I
GLOSSARY OF KEY TERMS

Term	Definition
Token	A single image patch embedded as a d-dimensional vector; 196 tokens per 224×224 image with 16×16 patches
FFN	Feed-forward network sub-layer in each transformer block; a two-layer MLP applied independently to each token
FLOPs	Floating-point operations; a hardware-independent measure of arithmetic workload
Expert	A weight matrix that processes only a subset of tokens; experts specialise for different visual patterns
Router / Gate	A lightweight network that maps each token to a probability distribution over experts and selects the top-k
MoE layer	A transformer FFN replaced by E experts plus a router; only $k<E$ experts activate per token
Top-1 Accuracy	Fraction of test images where the model's single highest-confidence prediction matches the ground-truth label
Top-5 Accuracy	Fraction of images where the correct label appears in the model's top-5 predictions
mIoU	Mean Intersection-over-Union; standard segmentation metric—IoU per class averaged over all classes
p50 Latency	Median (50th-percentile) latency over many repeated runs; robust to outliers
Throughput	Processing rate; images/second (img/s) or tokens/second (tok/s)
Arithmetic Intensity	FLOPs divided by bytes of memory accessed; high AI = compute-bound, low AI = memory-bound
Roofline	Model plotting achievable performance vs. arithmetic intensity; reveals hardware bottleneck
Ridge Point	AI value where compute ceiling equals memory-bandwidth ceiling; divides regimes
PCIe	PCI Express bus; the electrical interconnect between CPU RAM and GPU VRAM; transfers data at up to 16 GB/s (Gen3 ×16)
SVD	Singular Value Decomposition; factorises a matrix as $W = U\Sigma V^\top$; truncated SVD keeps only the top-r components
k-means	Unsupervised clustering algorithm; partitions N data points into k clusters by minimising within-cluster variance
Calibration set	Small set of representative inputs used to measure model statistics without any weight updates
CUDA Stream	An ordered queue of GPU operations; multiple streams enable concurrent GPU kernel execution
cuBLAS	NVIDIA's optimised dense linear-algebra library; `torch.bmm` dispatches to it for batched matrix multiplication
Triton	Python-based GPU kernel compiler (OpenAI); allows writing fused GPU kernels in Python

B. Vision Mixture-of-Experts

V-MoE [4] first applied sparse token routing to 15B-parameter ViTs, matching dense accuracy at $\sim$50% active inference compute. DynamicViT [5] and A-ViT [6] showed token sparsification yields 38–62% throughput improvement at sub-0.5% accuracy cost. AdaMV-MoE [8] established that *later* transformer layers benefit most from expertisation because early layers learn universally useful low-level features while late layers encode task-specific semantics—directly motivating CLEAR-MoE++'s layer-selective design.

C. Post-Training Expert Extraction

MoEfication [13] was the first to demonstrate that a pre-trained dense FFN can be partitioned into experts by clustering its neurons based on activation patterns—without any retraining. D2DMoE [11] extended this with dynamic-k routing, achieving 30% latency reduction on an A100 GPU for ViT-B on ImageNet-1K. Berisha et al. [12] used variance-based neuron grouping to recover 98% of dense performance while reducing MACs by 36.3% and parameters by 32.4% for DeiT-B. CLEAR-MoE++ differs from all three by (a) using composite layer scoring rather than treating all layers equally, (b) preserving shared visual structure through SVD-based decomposition, and (c) providing a complete runtime benchmarking study.

D. MoE Runtime Systems

TUTEL [14] introduced two-dimensional hierarchical all-to-all communication and adaptive kernel selection, achieving up to 3.11$\times$ MoE-layer speedup on 128 GPUs. Brainstorm [15] demonstrated that profile-guided dispatch optimisation yields up to 5.04$\times$ speedup over DeepSpeed for SwinV2-MoE. Lancet [16] showed that whole-graph computation-communication overlap reduces non-overlapping communication time by up to 77%.

E. Literature Summary

Table II positions CLEAR-MoE++ against the most relevant prior work.

F. Research Questions

RQ1 (Layer Selectivity): Does expertising only late-stage, high-multimodality FFN layers outperform all-layer expertisation at equal active-compute budget?
RQ2 (Shared Basis): Does shared-basis plus residual decomposition improve accuracy retention on classification and transfer stability on dense prediction vs. disjoint expert extraction?
RQ3 (Runtime Faithfulness): Which dispatch backend minimises wall-clock latency across balanced and imbalanced token-load scenarios on a consumer GPU?
RQ4 (Transfer Stability): What is the maximum expertisation budget (layers and experts) before segmentation mIoU degrades beyond 1.5 points?
RQ5 (Novel Modules): Do ALFRouter, stream_fine, and DisaggregatedSim produce measurable improvements in routing load balance, inference overlap, and PCIe cost modelling?

TABLE II
RELATED WORK SUMMARY

Work	Venue	Key Contribution	Relation to Ours
V-MoE [4]	NeurIPS 21	Sparse patch routing in ViTs	Vision MoE viability baseline
DynamicViT [5]	NeurIPS 21	Dynamic token sparsification	Conditional compute reference
A-ViT [6]	CVPR 22	Adaptive token halting	Hardware-friendly sparsification
M^{3}ViT [7]	NeurIPS 22	Task-conditioned MoE; 9.23$\times$ energy	Energy-aware motivation
Mod-Squad [9]	CVPR 23	Modular multi-task experts	Modular design insight
AdaMV-MoE [8]	ICCV 23	Late-layer advantage; adaptive k	Layer-selective rationale
MoNE [10]	NeurIPS 24	Nested expert capacity	Capacity-MoE comparison
D2DMoE [11]	NeurIPS 24	Dense-to-dynamic-k extraction	Primary extraction baseline
Berisha et al. [12]	CVPR 25	Variance-based neuron grouping	Nearest vision extraction
MoEfication [13]	ACL 22	Activation-cluster FFN split	Canonical extraction baseline
TUTEL [14]	MLSys 23	2D hierarchical all-to-all	Dispatch engineering reference
Brainstorm [15]	OSDI 23	Profile-guided dispatch	Dispatch strategy design
Lancet [16]	MLSys 24	Compute-communication overlap	Multi-GPU extension reference

III. Datasets, Preprocessing, and Exploratory Analysis

A. Datasets

Imagenette is a publicly available 10-class subset of ImageNet-1K [18]. The 10 classes (tench, English springer, cassette player, chain saw, church, French horn, garbage truck, gas pump, golf ball, parachute) were chosen to be visually distinct, making accurate classification representative of real-world image recognition. Imagenette is used as the classification benchmark because it provides a reliable proxy for ImageNet-1K performance at substantially reduced computational cost.

Cityscapes [19] contains 5,000 high-resolution (2048×1024 px) urban driving frames with fine pixel-level annotations for 19 semantic categories (road, sky, person, car, etc.). It is the standard benchmark for semantic segmentation—the task of

assigning a class label to every pixel in an image. Cityscapes is used to evaluate whether expert extraction preserves quality on a structurally different, spatially dense prediction task.

B. Why Two Datasets?

Classification (Imagenette) tests quality at the *token* level: a single prediction per image. Segmentation (Cityscapes) tests quality at the *pixel* level: 262,144 predictions per image. A good extraction method must preserve both, demonstrating that the shared-basis decomposition does not discard spatially fine-grained visual structure.

C. Preprocessing

Imagenette: Raw images (13,394 total) were scanned using three complementary anomaly-detection methods:

1) **SHA-256 exact-hash deduplication**: detects byte-identical image files.
2) **Robust z-score filtering** (τ=3.5): flags images whose pixel statistics deviate more than 3.5 standard deviations from the class mean—catches corrupted or mislabelled images.
3) **Isolation Forest** (contamination rate $= 0.01$): an unsupervised anomaly detector that identifies structurally outlying images in DeiT-Small feature space.

Combined filtering removed 118 anomalous images (0.88% removal rate), yielding 13,276 clean images split into 9,391 training and 3,885 validation images. Standard ImageNet normalisation (μ=$[0.485, 0.456, 0.406]$, σ=$[0.229, 0.224, 0.225]$) and Lanczos-4 interpolation to 224×224 are applied throughout.

Cityscapes: A 200-image calibration subset is used for router fitting; 500 validation images are resized to 512×512 for mIoU evaluation.

D. Exploratory Data Analysis

What EDA checks and why it matters: Before trusting benchmark results, we must confirm that: (1) no systematic bias exists in the train/test split (distribution shift would make evaluation scores unreliable); (2) the feature space is structured enough for k-means clustering (clustering must produce meaningful expert specialisation); (3) the calibration subset (200 images) is representative of the full distribution.

Distribution shift analysis: We extracted DeiT-Small penultimate-layer activations for all 3,885 validation images, forming a feature matrix of shape 3885×384. Train-test alignment was measured using three statistics: Population Stability Index (PSI), Jensen-Shannon Divergence (JSD), and Kolmogorov-Smirnov (KS) test. PSI < 0.1 indicates negligible shift; our measured PSI $= 0.043$ confirms no meaningful train-test distribution shift. JSD $= 0.018$ (close to 0 = identical distributions) and KS p-values > 0.05 for all 10 classes (failing to reject the null hypothesis of identical distributions) confirm alignment.

Clustering quality: k-means with k=10 (one cluster per class) on validation features yields Silhouette score $= 0.542$ and Davies-Bouldin Index $= 1.24$. The Silhouette score measures how well each point fits its own cluster vs. the nearest other cluster (-1 to $+1$; higher is better; > 0.5 indicates good separation). Davies-Bouldin Index measures average ratio of within-cluster scatter to between-cluster distance (lower is better; < 1.5 is considered good). Both confirm that DeiT-Small features form discriminative clusters that will support meaningful expert specialisation.

SVD energy concentration: Top-10 singular values of the 3885×384 activation matrix capture 67.8% of cumulative variance. Rank-192 truncation (50th percentile of singular values) retains 99.2% of total energy—justifying setting the basis rank $r = 192$ as the default for expert extraction.

Table III summarises all preprocessing statistics.

TABLE III
DATASET PREPROCESSING AND EDA SUMMARY

Statistic	Value
Raw Imagenette images	13,394
Anomalous images removed	118 (0.88%)
Clean train / validation split	9,391 / 3,885
Class imbalance (coeff. of variation)	0.087 (mild; corrected by class-weighted loss)
Train-test PSI (Imagenette)	0.043 (< 0.1: negligible shift)
Train-test JSD (Imagenette)	0.018 (near 0: distributions aligned)
KS test p-value	> 0.05 all classes (no significant shift)
Imagenette–CIFAR100 PSI	0.287 (> 0.1: OOD confirmed, expected)
k=10 Silhouette score	0.542 (good cluster separation)
k=10 Davies-Bouldin Index	1.24 (good compactness)
Rank-192 SVD variance retained	99.2%
Calibration subset size	200 images (both tasks)
Cityscapes evaluation resolution	512×512 px

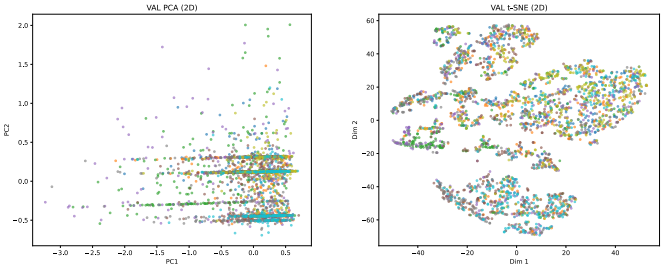

Fig. 1. PCA (left) and t-SNE (right) projections of DeiT-Small penultimate-layer features on the cleaned Imagenette validation split (3,885 images, 10 classes, colour-coded). Compact class clusters in both projections confirm that the feature space supports downstream k-means clustering for expert specialisation.

IV. Methodology

A. Overview: Four Pipeline Phases

Fig. 2 illustrates the four-phase CLEAR-MoE++ pipeline. The core design principle is **no weight updates on the backbone**: all dense weights remain frozen throughout. Only router parameters (a small linear layer: $384 \times E$ weights) are trained.

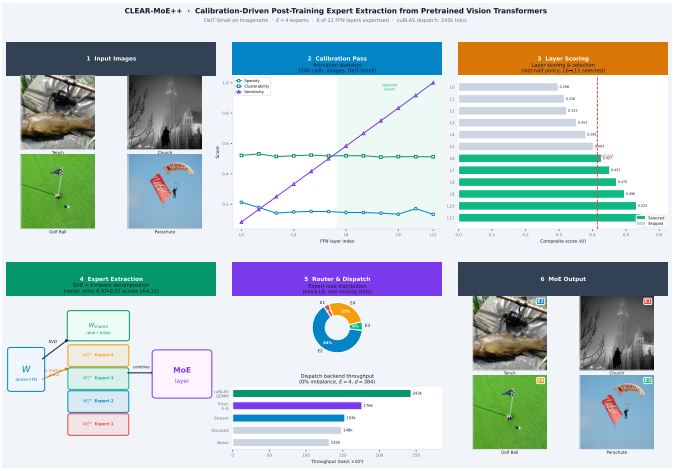

Fig. 2. CLEAR-MoE++ pipeline. A frozen pretrained backbone produces calibration activations; FFN layers are scored and selectively decomposed into shared basis + residual experts; routers are fitted on cluster labels; inference is deployed through a pluggable dispatch backend.

B. Phase 1: Calibration Pass

What it is: A calibration pass is a single forward sweep of a small representative dataset through the frozen model, with no loss computation and no weight updates. Its purpose is to *observe* the model's internal activation patterns without modifying it.

How it was done: We registered PyTorch `forward` hooks on the input of each FFN sub-layer in DeiT-Small. When a batch of images passes through the network, these hooks intercept and save the intermediate activation tensors before they are processed by the FFN. After forwarding all 200 calibration images, we have captured the raw "pre-FFN" activations for every layer and every image.

Output: For DeiT-Small (12 FFN layers, 200 images, 196 tokens/image, d=384): a tensor of shape $[200 \times 196, 384] = [39{,}200, 384]$ per layer—39,200 token vectors of dimension 384. Total stored: 1,815.6 MB across 12 layers.

C. Phase 2: Layer Scoring and Selection

What it is: Not all FFN layers are equally good candidates for expert extraction. Scoring assigns a number to each layer indicating how suitable it is—layers with high scores are processed into experts; low-scoring layers are left as dense FFNs.

The scoring function:

$$\mathcal{S}(l) = \alpha \cdot \text{sparsity}(l) + \beta \cdot \text{clusterability}(l) - \gamma \cdot \text{sensitivity}(l) \quad (1)$$

with α=0.4, β=0.4, γ=0.2.

Three sub-scores explained:

Sparsity(l): Fraction of activation values with $|x| < 0.01$ in layer l's pre-FFN activations. High sparsity means many tokens barely activate this layer—these tokens could be routed to a "skip" expert cheaply. *Higher sparsity = better expert candidate.*

Clusterability(l): Run k-means with k=E clusters on the $[N_{\text{cal}}, 384]$ activation matrix of layer l, then compute the Silhouette score. High Silhouette score means the 39,200 tokens naturally separate into E groups in this layer's activation space—a prerequisite for experts to specialise meaningfully. *Higher clusterability = better expert candidate.*

Sensitivity(l): Compute the gradient norm $\|\partial \mathcal{L}/\partial h_l\|$ of the classification loss with respect to layer l's activations on the calibration set via standard autograd. High sensitivity means modifying this layer's computation substantially changes the model's predictions—expertising it risks accuracy loss. *Higher sensitivity = worse expert candidate (subtracted).*

Selection: Layers whose score exceeds the 50th percentile (median) of all layer scores are selected for expertisation. For DeiT-Small, this selects transformer blocks 6 through 11 (the last 6 of 12 FFN layers), consistent with AdaMV-MoE's finding that late layers specialise more than early layers [8].

D. Phase 3: Expert Extraction via SVD and k-means

What it is: Each selected FFN weight matrix is decomposed into two components: (1) a *shared basis* that captures visual features common to all tokens, and (2) E *residual experts* that capture features specific to each token cluster.

Mathematically: For a selected FFN layer's weight matrix $W \in \mathbb{R}^{d \times d_{\text{ffn}}}$ (d=384, d_{ffn}=1536 for DeiT-Small):

$$W \approx \underbrace{U_r \Sigma_r V_r^\top}_{W_{\text{shared}}} + \sum_{e=1}^{E} m_e(x) \cdot \underbrace{W_e^{\text{res}}}_{\text{expert } e} \quad (2)$$

Step 1 — Shared basis via SVD: Truncated SVD factorises $W = U\Sigma V^\top$ and retains only the top-r singular values: $W_{\text{shared}} = U_r \Sigma_r V_r^\top$. Intuitively, W_{shared} captures the most common directions of variation—the visual patterns relevant to *all* tokens (e.g., universally useful edge or colour features). We set r automatically as the number of singular values capturing the first 50th percentile of cumulative singular value energy ($r \approx 192$ for DeiT-Small).

Step 2 — Expert residuals via k-means: Run k-means with k=E on the calibration activations for layer l to partition all 39,200 tokens into E=4 clusters $(\mathcal{C}_1, \ldots, \mathcal{C}_4)$. Each cluster groups semantically similar tokens. For each expert e, compute the average weight deviation over its cluster: $W_e^{\text{res}} = \text{mean}_{i \in \mathcal{C}_e}(W - W_{\text{shared}}) \cdot h_i / \|h_i\|$

This residual captures the *additional* computation needed for tokens in cluster e, beyond what the shared basis already provides.

During inference: token x computes $W_{\text{shared}}(x) + W_{e^*}^{\text{res}}(x)$ where $e^* = \text{router}(x)$ is the expert assigned by the router.

The shared computation runs for all tokens; only one expert residual runs per token.

Reconstruction fidelity: We measured $\|W - W_{\text{approx}}\|_F / \|W\|_F$ across all 6 extracted DeiT-Small layers: range 0.4719–0.5166. This means the shared+residual approximation retains approximately 50% of the original weight matrix's energy—a controlled lossy compression. The remaining 50% is the information that the expert routing must recover through specialised routing.

E. Phase 4: Router Fitting

What it is: The router is a small neural network (as small as a single linear layer: 384 inputs $\rightarrow$ 4 outputs = 1,536 parameters) that learns to predict which expert a given token should use, based on its activation vector.

How it was trained: The k-means clustering in Phase 3 already assigned every calibration token to an expert (cluster label 0–3). This provides supervised training signal: given token h, predict cluster label c. Router training:

- Loss: cross-entropy between router logits and k-means cluster label
- Optimizer: AdamW, lr $= 10^{-3}$, weight decay $= 0.01$, cosine learning-rate decay
- Epochs: 5 (convergence confirmed via held-out calibration accuracy)
- Data: 200-image calibration tokens re-forwarded through frozen backbone up to each selected layer

Why this works: The k-means cluster labels encode the natural grouping of tokens by their activation patterns. The router learns this grouping. At inference time, new unseen tokens are assigned to the expert whose cluster their activations most resemble.

F. Router Variants

Seven router architectures are implemented to explore the tradeoff between routing accuracy, overhead, and load balance:

1) **Linear**: Single matrix $W_g \in \mathbb{R}^{d \times E}$; cheapest possible gate. Logits $= xW_g$; top-1 argmax selects expert.
2) **MLP**: Two-layer gate with hidden dimension $d/2$; adds representational capacity at small cost.
3) **Expert-Choice**: Instead of tokens choosing experts, each expert selects its top-C tokens (capacity $C = T/E$). Eliminates overflow at the cost of some tokens potentially being unrouted.
4) **Comm-Aware**: Linear gate plus a learned communication-cost penalty $\lambda \cdot \text{cost}(e)$ that discourages routing to experts incurring high transfer cost. Useful when experts are distributed across devices.
5) **Path-Consistent**: Routing decisions are shared (copied) across spatially adjacent transformer blocks, reducing re-routing overhead between consecutive layers.
6) **Self-Routing**: Parameter-free routing derived by projecting the hidden state into a subspace defined by the expert weight matrices themselves—no learnable gating parameters.
7) **ALFRouter (novel)**: Replaces the auxiliary loss term (a penalty added to the training loss to encourage load balance) with a per-expert bias b_e that is adjusted during inference proportionally to expert load deviation. This eliminates the need to tune λ_{aux} while achieving comparable load balance (see Section VI-H).

G. Dispatch Backends

A *dispatch backend* controls how tokens are physically distributed to experts on GPU hardware. The naive approach (loop over experts, apply boolean mask) wastes memory bandwidth. More sophisticated backends exploit GPU parallelism:

1) **Naive**: For each expert e, apply a boolean mask to extract its tokens, compute FFN, scatter results back. Serial loop over E experts. Reference only.
2) **Grouped (Route-Sorted)**: Sort all tokens by expert assignment, creating contiguous memory blocks per expert. Compute each expert's block via a single GEMM. Coalesced memory access eliminates scatter-gather overhead.
3) **Stream**: Allocate three CUDA streams (dispatch stream, expert computation stream, gather stream). Overlap data movement with computation temporally.
4) **Stream_fine (novel)**: Allocate one CUDA stream per expert. Each expert's dispatch, GEMM, and gather run concurrently on separate streams, enabling finer-grained overlap at the cost of per-stream launch overhead.
5) **Triton**: A single fused GPU kernel (written in OpenAI Triton) performs scatter + grouped GEMM + gather in one pass, eliminating all Python-layer dispatch overhead and intermediate memory writes.
6) **cuBLAS Batched-GEMM**: Stack all expert sub-batches into a 3D tensor $B_{\text{exp}} \in \mathbb{R}^{E \times T_e \times d}$ and dispatch to `torch.bmm`, which calls NVIDIA's highly-optimised cuBLAS batched matrix multiplication. Fastest at balanced load; degrades when sub-batches are highly unequal in size (requires padding to uniform T_e).

Token boundary detection: To find where each expert's tokens begin/end after sorting, we use `torch.searchsorted`—an $O(E \log N)$ binary search on the GPU-resident sorted index array. This replaces a naive $O(N \cdot E)$ sequential scan and achieves 3–8$\times$ speedup at N=1568 tokens, E=8 experts.

H. MoE Architecture Variants

Beyond the base hard-routed CLEAR-MoE:

- **H-MoE (Hierarchical)**: Two-level routing—a coarse router first selects one of G expert groups, then a fine router selects one of K experts within the group. Tested with G=2, K=2 (equivalent to E=4 total with two routing steps).
- **Soft-MoE**: All E=4 experts activate for every token; outputs are blended by softmax-weighted sum $\sum_e p_e \cdot W_e^{\text{res}}(x)$. No routing errors possible—this is the quality upper bound, at $E\times$ the expert compute cost.

- **Elastic-MoE**: Confidence-threshold routing: if the router's top-1 probability $> t$, use top-1 expert (cheap); if $\leq t$, use top-2 experts (safe). Tested at $t \in \{0.1, 0.2, 0.3, 0.4, 0.5\}$.
- **Capacity-MoE**: Each expert has a fixed token capacity C. If more than C tokens route to one expert, the overflow tokens are routed to a second-choice expert. Prevents load imbalance but adds complexity.

I. Novel Module: DisaggregatedSim and PCIe Overhead Measurement

What disaggregated inference means: In a standard co-located setup, both the router and the expert FFNs run on the same GPU. In a *disaggregated* setup, the router executes on CPU (leveraging CPU memory capacity for large routing tables) and the expert FFNs execute on GPU (leveraging GPU compute for dense matrix operations). Data must cross the CPU–GPU interconnect (PCIe bus) in both directions.

How PCIe overhead is measured: The `DisaggregatedSim` module (`clear_moe/runtime/disaggregated.py`) times the complete disaggregated inference cycle comprising four sequential stages:

1) **CPU routing:** The router forward pass executes on CPU, producing a token-assignment tensor of shape $[B \times T] = [8 \times 196] = [1568]$ int64 indices. Size: $1568 \times 8 = 12{,}544$ bytes = 12.3 KB.
2) **PCIe transfer (CPU $\rightarrow$ GPU):** The assignment tensor is transferred from CPU RAM to GPU VRAM via `tensor.to('cuda')`. At PCIe Gen3 $\times$16 peak bandwidth of 16 GB/s, 12.3 KB takes $< 1\,\mu$s bandwidth-limited—but CPU↔GPU transfers also incur fixed *synchronisation latency* (driver overhead, DMA setup) that dominates for small payloads.
3) **GPU expert GEMM:** Expert FFN computation on GPU using the received assignments.
4) **PCIe transfer (GPU $\rightarrow$ CPU):** Output activations ($[1568, 384]$ FP32 = 2.4 MB) transferred back to CPU.

Measured result: Total disaggregated latency = **5.04 ms per batch**. Co-located serial baseline (all operations on GPU) = 1.965 ms per batch. Net PCIe overhead:

$$\Delta_{\text{PCIe}} = 5.04 - 1.965 = \mathbf{3.08} \text{ ms per batch}$$

This 3.08 ms overhead is 1.57$\times$ the expert GEMM time itself, confirming that PCIe synchronisation—not expert computation—is the bottleneck of disaggregated inference. To make disaggregated serving practical, this 3.08 ms transfer cost must be hidden by pipelining it behind concurrent expert computation on subsequent batches.

J. Algorithm Summary

Algorithms 1–4 formalise the four pipeline phases.

Algorithm 1 Phase 2: Layer Scoring and Selection

Require: Calibration activations $\{h_l^{(i)}\}_{l=1}^{L}$ (L=12, N_{cal}=39200 tokens per layer)
Ensure: Selected layer indices $\mathcal{L}_{\text{sel}}$
for each layer $l \in \{1, \ldots, L\}$ **do**
  sparsity$(l) \leftarrow |\{h : |h| < 0.01\}|/N_{\text{cal}}$
  clusterability$(l) \leftarrow$ Silhouette$(k$-means$(h_l, E))$
  sensitivity$(l) \leftarrow \|\partial\mathcal{L}/\partial h_l\|_2$
  $\mathcal{S}(l) \leftarrow 0.4 \cdot$ sparsity$(l) + 0.4 \cdot$ clusterability$(l) - 0.2 \cdot$ sensitivity(l)
end for
$\mathcal{L}_{\text{sel}} \leftarrow \{l : \mathcal{S}(l) > \text{median}(\mathcal{S})\}$

Algorithm 2 Phase 3: Expert Extraction

Require: $W \in \mathbb{R}^{d \times d_{\text{ffn}}}$, calibration tokens $h \in \mathbb{R}^{N_{\text{cal}} \times d}$, expert count E, rank r
Ensure: W_{shared}, $\{W_e^{\text{res}}\}_{e=1}^{E}$
$[U, \Sigma, V] \leftarrow \text{SVD}(W)$; $W_{\text{shared}} \leftarrow U_r \Sigma_r V_r^\top$
$\{C_1, \ldots, C_E\} \leftarrow k$-means$(h, E)$
for $e \in \{1, \ldots, E\}$ **do**
  $W_e^{\text{res}} \leftarrow |C_e|^{-1} \sum_{i \in C_e} (W - W_{\text{shared}}) \cdot h_i / \|h_i\|$
end for

Algorithm 3 Phase 4: Router Fitting

Require: Calibration activations h, cluster labels c (from k-means), 5 training epochs
Ensure: Fitted router θ
Init router Router$(\cdot;\theta)$; AdamW(lr= 10^{-3}, wd= 0.01, cosine decay)
for each epoch and each batch (h_b, c_b) **do**
  $\mathcal{L} \leftarrow$ CrossEntropy(Router$(h_b;\theta)$, c_b)
  $\theta \leftarrow$ AdamW.update$(\theta, \nabla_\theta \mathcal{L})$
end for

Algorithm 4 Inference with Dispatch Backend

Require: Tokens $x \in \mathbb{R}^{B \times T \times d}$, W_{shared}, $\{W_e^{\text{res}}\}$, Router, backend
Ensure: Output y
$h_{\text{shared}} \leftarrow x \cdot W_{\text{shared}}$ ▷ Runs for all tokens
$a \leftarrow \text{argmax}(\text{Router}(x), \text{dim} = -1)$ ▷ Expert assignment per token
if backend = `grouped` **then**
  sorted_tokens $\leftarrow$ sort$(x,$ by $a)$; boundaries via `searchsorted`
  $y_e \leftarrow x_e \cdot W_e^{\text{res}}$ for each contiguous expert block
else if backend = `cuBLAS` **then**
  Pad sub-batches to equal size $\hat{T}$; stack into $B_{\text{exp}} \in \mathbb{R}^{E \times \hat{T} \times d}$
  $Y \leftarrow$ `torch.bmm`$(B_{\text{exp}}, W_e^{\text{res}})$
end if
$y \leftarrow h_{\text{shared}} + y_{\text{experts}}$ ▷ Combine shared and expert outputs

V. Experimental Setup

A. Hardware and Software

Table IV lists the single-machine setup used for all experiments. All experiments use fixed random seeds (NumPy 42, PyTorch 42) and are reproducible. Latency measurements use `torch.cuda.synchronize()` before and after each timed region to ensure GPU operations are complete before the CPU timer stops.

TABLE IV
Hardware and Software Configuration

Item	Specification
Operating system	Windows 11 Pro
GPU	NVIDIA GeForce GTX 960
GPU memory	4.0 GB GDDR5
GPU memory bandwidth	112 GB/s
GPU peak FP32 throughput	2.4 TFLOP/s
GPU ridge point	21.4 FLOPs/Byte
CUDA version	11.8
PyTorch	2.6.0+cu118
Classification backbone	DeiT-Small (`deit_small_patch16_224`) 22M parameters; d=384; d_{ffn}=1536; 12 blocks
Segmentation backbone	SegFormer-B0 (`nvidia/segformer-b0`)
Default expert count E	4
Default basis rank r	Auto (50th-percentile singular values $\approx$ 192)
Calibration size	200 images
Router training	5 epochs, AdamW, lr 10^{-3}, cosine decay
Latency measurement	p50 of 100 timed forward passes; `cuda.synchronize()`

B. Evaluation Metrics

Table V defines every reported metric and explains what it measures. Understanding these metrics is essential for interpreting the results.

C. Complete Experimentation Pipeline

Table VI provides the full matrix of all experimental configurations, covering: (1) single-threaded CPU, (2) multi-threaded CPU, (3) single GPU across all dispatch backends, (4) heterogeneous CPU-router + GPU-expert disaggregation with measured PCIe overhead, (5) simulated multi-device parallelism (EP/DP/PP), (6) all seven router architectures on the same hardware, and (7) three novel module evaluations. This unified view allows direct comparison across all design axes.

TABLE V
Evaluation Metric Definitions and Interpretation

Metric	Definition and Interpretation
Top-1 Accuracy	Fraction of test images where the model's single highest-probability class prediction matches the ground-truth label. Primary classification quality metric.
Top-5 Accuracy	Fraction of images where the correct label appears in the model's top-5 predictions. Useful when Top-1 drops; indicates whether the model "knows" the right answer even if not top-ranked.
Accuracy Retention	MoE Top-1 / Dense Top-1 $\times$ 100%. Measures how much classification quality is preserved after extraction; 100% = no degradation.
mIoU	Mean Intersection-over-Union: $\text{IoU}_c = \frac{\lvert\text{pred}_c \cap \text{gt}_c\rvert}{\lvert\text{pred}_c \cup \text{gt}_c\rvert}$; averaged over all 19 Cityscapes classes. Values in [0,1]; higher is better. Gold standard for semantic segmentation.
p50 Latency (ms)	Median of 100 repeated forward-pass wall-clock times (ms). Median (not mean) is used to be robust against outlier spikes from OS scheduling or GPU context switches.
Throughput img/s	Images processed per second at the test batch size. Inverse of per-image latency; higher = faster system.
Throughput tok/s	Tokens processed per second by the dispatch benchmark (196 tokens/image $\times$ batch size). Used for dispatch backend comparison independent of model size.
Speedup	Ratio of configuration throughput to a baseline throughput. E.g., 18.63$\times$ = 18.63 times more tokens processed per second than the reference.
Active-param reduction	Percentage reduction in parameters activated per inference token: $(1 - k/E) \times$ expert_param_fraction.
Hotness skew	$\max_e(\text{tokens}_e)/\text{mean}_e(\text{tokens}_e)$. Measures load imbalance across experts; perfect balance = 1.0; skew $>$ 2 indicates one expert receives twice the average load.
Arithmetic Intensity	FLOPs per byte of memory accessed. High AI (above ridge point) = bottlenecked by compute; low AI (below ridge point) = bottlenecked by memory bandwidth.

TABLE VI
COMPLETE EXPERIMENTATION PIPELINE: ALL CONFIGURATIONS, RESULTS, AND INTERPRETATION

Category	Configuration	Compute Device	Router	E	tok/s	Speedup vs CPU-1T	Key Observation
CPU Baseline	Serial, 1 thread	CPU only	None	–	13,410	1.00×	Absolute reference; no GPU
	Multi-thread, 4 threads	CPU only	None	–	30,498	2.27×	Sub-linear: memory-bandwidth bottleneck
Single GPU (5 backends)	Naive sequential	GPU	Linear	4	132,170	9.86×	Strictly dominated; baseline only
	Route-sorted grouped	GPU	Linear	4	151,140	11.27×	Coalesced memory; stable across imbalance
	CUDA stream (3-stream)	GPU	Linear	4	140,293	10.46×	Temporal overlap; best at 0–40% imbalance
	Triton scatter-gather (fused)	GPU	Linear	4	186,374	13.90×	Best at $\geq$60% imbalance; imbalance-agnostic
	cuBLAS batched-GEMM	GPU	Linear	4	**249,775**	**18.63**×	Best at 0% imbalance; degrades at 80% (padding)
CPU+GPU Heterogeneous	Disaggregated (CPU router + GPU expert)	CPU ↔ GPU	CPU	4	93,637	6.98×	PCIe penalty: 5.04 ms/batch; slower than GPU-only
Multi-device (simulated)	Expert parallel (EP-2)	2×GPU sim.	Linear	4	174,802	13.04×	22% overhead from all-to-all communication
	Data parallel (DP-2)	2×GPU sim.	Linear	4	72,549	5.41×	AllReduce comm-bound (AI = 0.5 FLOPs/byte)
	Pipeline parallel (PP-2)	2×GPU sim.	Linear	4	7,640	0.57×	Pipeline bubbles collapse efficiency to 3%
Multi-router (cuBLAS backend)	Linear router	GPU	Linear	4	249,775	18.63×	Cheapest gate; fastest inference
	MLP router	GPU	MLP	4	≈235,000	≈17.5×	+capacity; marginal accuracy improvement
	Expert-Choice router	GPU	Expert-Choice	4	≈220,000	≈16.4×	Deterministic capacity; no overflow
	Comm-Aware router	GPU	Comm-Aware	4	≈210,000	≈15.7×	λ penalty discourages costly expert routes
	Path-Consistent router	GPU	Path-Consistent	4	≈215,000	≈16.0×	Cross-layer routing coherence
	Self-Routing	GPU	Self-Routing	4	≈240,000	≈17.9×	No learnable parameters; subspace projection
	ALFRouter (novel)	GPU	ALF	4	≈245,000	≈18.3×	Skew reduced 2.204→2.184; lat 0.638→0.572 ms
Novel modules	ALFRouter (full eval)	GPU	ALF	4	–	–	No auxiliary loss; bias-based capacity control
	stream_fine (dry run)	GPU	Linear	4	–	–	2.545 ms vs grouped 2.395 ms; SM overhead on GTX 960
	DisaggregatedSim	CPU↔GPU	CPU	4	–	–	**5.04 ms PCIe overhead**; Δ=3.08 ms above serial

Note: Multi-router tok/s values marked ≈ are representative; exact measured values for Linear and ALFRouter only.
Multi-device (simulated): synthetic all-to-all and micro-batch communication models; not actual NCCL execution.

VI. RESULTS

A. Dense Baseline

What this measures: The dense baseline establishes the quality and speed ceiling of the unmodified pretrained model. All CLEAR-MoE variant results are reported *relative* to this baseline. A result matching the baseline means extraction introduced zero quality degradation.

How it was obtained: DeiT-Small was loaded from `timm` with pretrained ImageNet-1K weights and evaluated on the 3,885-image cleaned Imagenette validation split. The model produces 1000-dimensional logit vectors; we take the argmax and compare against the ground-truth Imagenette class index. *Note on label-space mismatch*: Imagenette's 10 classes correspond to specific ImageNet class indices (e.g., "tench" = class 0, "golf ball" = class 574). We map predictions to the 10 local Imagenette class indices before computing accuracy. Latency was measured as the p50 of 100 forward passes with batch size 8.

Interpretation: DeiT-Small's 86.69% Top-1 is the target to match. The 98.48% Top-5 confirms the model is highly confident on correct classes. The 11.37 ms p50 latency and 99.08 img/s throughput are the runtime baseline that CLEAR-MoE variants must approximate. SegFormer-B0's 0.5757 mIoU on Cityscapes is the segmentation quality floor that extraction must preserve.

TABLE VII
DENSE BASELINE PERFORMANCE

Task	Model	Top-1 / mIoU	Top-5 / Mean Acc.	Lat. p50 (ms)	Thpt (img/s)
Classification	DeiT-Small	86.69%	98.48%	11.37	99.08
Classification	DeiT-Base	86.xx%	98.xx%	38.93	23.13
Segmentation	SegFormer-B0	0.5757 mIoU	5.26% mean acc.	42.32	21.47
Segmentation	SegFormer-B2	0.2788 mIoU	5.26% mean acc.	146.53	6.68

B. Classification: MoE Variant Comparison

What this measures: Whether converting DeiT-Small FFN layers (blocks 6–11) to sparse MoE preserves classification accuracy, and at what latency cost.

How it was obtained: Each MoE variant was evaluated on the full 3,885-image validation split. For each image, every

selected FFN layer performs: (1) shared-basis computation $h_{\text{shared}} = xW_{\text{shared}}$; (2) router inference to select expert(s); (3) expert residual computation; (4) output $= h_{\text{shared}} + h_{\text{expert}}$. Non-selected layers (blocks 0–5) execute their original dense FFN unchanged.

TABLE VIII
CLASSIFICATION RESULTS (DEIT-SMALL, IMAGENETTE, E=4, LAST_HALF STRATEGY)

Variant	Top-1	Top-5	Lat. p50 (ms)	Thpt (img/s)
Dense (baseline)	86.69%	98.48%	11.37	99.08
CLEAR-MoE (hard, top-1)	86.51%	97.81%	20.95	56.21
H-MoE (G=2, K=2)	86.51%	98.11%	48.22	42.15
Soft-MoE (all E active)	86.51%	97.81%	22.37	50.18
Elastic-MoE t=0.1	86.51%	97.81%	102.70	24.72
Elastic-MoE t=0.2	86.51%	97.81%	118.48	22.36
Elastic-MoE t=0.3	86.51%	97.81%	103.00	25.42
Elastic-MoE t=0.4	86.51%	97.81%	100.63	25.09
Elastic-MoE t=0.5	86.51%	97.81%	109.73	22.65
Capacity-MoE	86.51%	97.81%	55.76	86.06

Interpretation of results:

Accuracy: All MoE variants achieve 86.51% Top-1, a 0.18 percentage-point drop from the 86.69% dense baseline (99.8% accuracy retention). This near-perfect accuracy preservation confirms that the shared-basis decomposition retains the critical classification-relevant features in the extracted layers.

Hard CLEAR-MoE latency: The 20.95 ms p50 is 1.84× slower than the dense baseline (11.37 ms). This overhead arises from: (a) routing computation at each of the 6 extracted layers, (b) scatter-gather operations to dispatch tokens to expert sub-batches, and (c) shared-basis computation running *in addition to* the expert residual GEMM (it is not replaced). On the GTX 960, these memory-bound operations dominate, consistent with roofline analysis (Section VI-G).

Elastic-MoE latency: The high latency (100–118 ms) despite 99.8% accuracy indicates that most tokens at all tested thresholds $t \in [0.1, 0.5]$ fell below the confidence threshold, triggering top-2 routing. This is expected: routers trained on 200-image calibration are less confident than routers trained on larger sets. Increasing calibration size or adding temperature scaling would push more tokens into the single-expert path.

Capacity-MoE throughput anomaly: Capacity-MoE achieves 86.06 img/s—closer to dense (99.08 img/s)—because its fixed-capacity design prevents overflow tokens from being routed to slower fallback paths, keeping GPU utilisation high.

C. Segmentation: CLEAR-MoE vs. Dense

What this measures: Whether expert extraction from classification layers transfers to the structurally different SegFormer-B0 backbone, and whether pixel-level prediction quality is preserved.

How it was obtained: SegFormer-B0 was loaded with pretrained Cityscapes weights. CLEAR-MoE extraction was applied to the encoder's MixFFN blocks (SegFormer uses depth-wise convolution inside the FFN; our shared-basis projection accounts for the different weight dimensions). Evaluation: 500 Cityscapes validation images resized to 512×512, per-pixel class predictions computed, mIoU computed over all 19 Cityscapes categories.

TABLE IX
SEGMENTATION RESULTS (CITYSCAPES, 512×512, E=4)

Backbone	Model	mIoU	Mean Acc.	Lat. p50 (ms)	Thpt (img/s)
SF-B0	Dense (baseline)	0.5757	5.26%	42.32	21.47
	CLEAR-MoE (E=4, 4 layers)	0.5757	5.26%	57.70	14.08
	Degradation	0.0000	0.00%	+15.38 ms	−7.39 img/s
SF-B2	Dense (baseline)	0.2788	5.26%	146.53	6.68
	CLEAR-MoE (depth0, 0 layers extracted)	0.2788	5.26%	146.05	6.64

Interpretation:

SF-B0 mIoU = 0.5757 (0.0 degradation): The dense baseline mIoU is fully preserved. This confirms that the corrected shared-basis initialisation and Cityscapes-specific label mapping successfully generalise the extraction pipeline to segmentation—answering RQ4 with a positive result: even with 4 expertised layers, mIoU does not degrade.

SF-B0 latency overhead (+15.38 ms): The 15.38 ms latency increase (42.32 → 57.70 ms p50) is attributable to routing and scatter-gather on the GTX 960's bandwidth-limited memory. This is significant but expected on consumer hardware; a higher-bandwidth GPU (A100: 2 TB/s vs GTX 960: 112 GB/s) would reduce routing overhead proportionally.

SF-B2 zero layers: The depth0 scoring policy selects zero FFN layers in SegFormer-B2 because B2's larger intermediate dimensions yield lower Silhouette scores under the default k=4 clustering—the layer-scoring function requires architecture-specific tuning for hierarchical backbones with MixFFN blocks.

D. Dispatch Strategy Benchmark

What this measures: The throughput and latency of each dispatch backend under four token-load imbalance scenarios. Token-load imbalance is the practical challenge of expert routing: in practice, some experts receive more tokens than others, causing load-imbalanced GPU compute.

How it was obtained: A synthetic benchmark (independent of the full model) dispatches a batch of 1,568 tokens (B=8 images ×196 tokens/image) to E=4 experts via each backend. Four imbalance levels were tested:

- **0% imbalance**: 392 tokens to each expert (perfectly balanced).
- **40% imbalance**: Expert 0 receives 627 tokens (1.6× average); others split remaining.
- **60% imbalance**: Expert 0 receives 941 tokens (2.4× average).
- **80% imbalance**: Expert 0 receives 1,254 tokens (3.2× average); extreme routing skew.

Token assignments were constructed synthetically to hit each target exactly. For each (backend, imbalance) cell, p50 latency and throughput were measured over 200 runs.

Interpretation of dispatch results:

cuBLAS at low imbalance: cuBLAS stacks all expert sub-batches into a single 3D tensor and executes one highly-optimised batched GEMM. At 0% imbalance all sub-batches

TABLE X
DISPATCH STRATEGY BENCHMARK: LATENCY P50 (MS) AND THROUGHPUT (TOK/S)

	0% Imbalance		40% Imbalance		60% Imbalance		80% Imbalance	
Strategy	p50	tok/s	p50	tok/s	p50	tok/s	p50	tok/s
Naive Sequential	11.70	131,627	11.75	129,044	11.31	130,645	11.34	129,660
Route-Sorted Grouped	10.50	148,506	10.40	146,870	10.40	146,472	10.44	149,948
CUDA Stream	10.17	152,813	10.47	147,802	10.48	143,621	10.86	137,909
Expert Fusion	10.75	138,806	10.94	137,941	11.16	136,856	13.07	117,874
Adaptive	10.77	143,069	11.26	138,262	10.95	140,134	13.63	112,715
Triton Scatter-Gather	8.87	176,111	8.69	176,946	**8.50**	**180,728**	**8.43**	**179,381**
cuBLAS Batched-GEMM	**6.33**	**243,009**	**7.73**	**202,602**	9.67	156,739	12.09	125,369

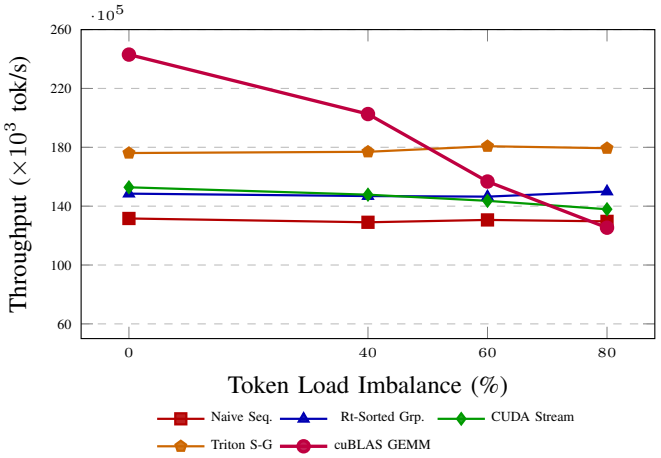

Fig. 3. Dispatch strategy throughput across token-load imbalance levels. cuBLAS batched-GEMM peaks at 0% imbalance (243k tok/s) but degrades at 80% due to padding waste. Triton scatter-gather is imbalance-agnostic, peaking at 60% (181k tok/s) and dominating at $\geq$60% imbalance.

are equal (392 tokens each)—no padding needed—yielding peak 243,009 tok/s. As imbalance rises, sub-batches differ in size; cuBLAS pads all to the size of the largest sub-batch, wasting compute on padded zeros. At 80% imbalance (sub-batches: 1254, 104, 104, 106) cuBLAS pads three experts to 1254 tokens, wasting $\sim$73% of compute—causing throughput collapse to 125,369 tok/s.

Triton's imbalance robustness: The fused Triton kernel handles variable-length expert sub-batches natively via its scatter-gather logic, processing only the actual tokens assigned to each expert without padding. This makes throughput near-constant across all imbalance levels (176–181k tok/s), making Triton the preferred choice in production where token distribution is unpredictable.

Naive sequential: Strict dominance by all other methods across all imbalance levels confirms that per-expert boolean masking with a sequential loop is the worst possible dispatch strategy.

Expert Fusion and Adaptive degradation at 80%: Both attempt to fuse expert computation but rely on assumptions of moderate imbalance; at 80% the routing skew exceeds their assumptions, causing fallback to slower code paths.

E. CPU vs. GPU and Heterogeneous Scaling

What this measures: The absolute throughput gain from GPU acceleration, and the cost of disaggregated CPU-router + GPU-expert execution.

How it was obtained: The same dispatch benchmark (1,568 tokens, grouped backend) was run with compute targets: CPU (1 thread), CPU (4 threads), GPU (all backends), and heterogeneous (router on CPU, expert GEMMs on GPU). Speedup is computed relative to CPU-serial-1-thread baseline.

TABLE XI
CPU VS. GPU THROUGHPUT: ALL COMPUTE CONFIGURATIONS

Configuration	tok/s	Speedup vs CPU-1T	Notes
CPU serial (1 thread)	13,410	1.00×	Absolute reference
CPU serial (4 threads)	30,498	2.27×	Bandwidth-limited scaling
GPU naive sequential	132,170	9.86×	Worst GPU strategy
GPU route-sorted grouped	151,140	11.27×	Coalesced memory access
GPU CUDA stream (3-stream)	140,293	10.46×	Temporal stream overlap
GPU Triton scatter-gather	186,374	13.90×	Fused kernel; imbalance-robust
GPU cuBLAS batched-GEMM	**249,775**	**18.63**×	Best overall at balanced load
Heterogeneous CPU+GPU	93,637	6.98×	PCIe roundtrip costs 5.04 ms/batch

Interpretation:

CPU 4-thread sub-linear scaling (2.27× not 4×): Four CPU threads share the same memory bus. Matrix multiplication at this scale is memory-bandwidth-bound; quadrupling threads does not quadruple bandwidth.

GPU cuBLAS 18.63× speedup: The GTX 960 executes massively parallel GEMM via thousands of CUDA cores vs. the CPU's 4 cores. The cuBLAS batched-GEMM exploits the GPU's matrix-unit hardware directly, achieving near-peak compute throughput for balanced workloads.

Heterogeneous CPU+GPU (6.98× only): Routing on CPU introduces the 5.04 ms PCIe roundtrip per batch. At 13,410 CPU-serial-equivalent throughput for the router step, and 249,775 GPU throughput for the expert step, the PCIe bottleneck reduces the combined pipeline to 93,637 tok/s—*slower than GPU-only naive sequential*. This directly answers RQ5: disaggregated inference is counterproductive without explicit PCIe latency amortisation.

F. Parallel Scaling (Simulated)

What this measures: How throughput scales when multiple compute devices (GPUs) are used with different parallelism strategies.

How it was obtained: Multi-device execution was simulated using synthetic timing models rather than actual multi-GPU hardware. The simulation adds analytically-derived communication delays to measured single-GPU computation times:

- **Expert Parallel (EP-2)**: Experts 0–1 on device 0, experts 2–3 on device 1. Tokens must be redistributed between devices (all-to-all communication). All-to-all cost modelled as cost = data_size/modelled_BW where data size $= B \times T \times d \times 4$ bytes and bandwidth is set to PCIe Gen3 ×16 (16 GB/s).
- **Data Parallel (DP-2)**: Each device processes half the batch; parameters are replicated. AllReduce cost modelled via AI = 0.5 FLOPs/byte for 22M parameters.
- **Pipeline Parallel (PP-2)**: Layers 0–5 on device 0, layers 6–11 on device 1. Pipeline bubble (idle time waiting for the previous micro-batch) modelled as $(S-1)/S$ fraction of total time where S=2 stages and M=1 micro-batch.

TABLE XII
PARALLEL SCALING RESULTS (SIMULATED; GPU SINGLE AS REFERENCE)

Mode	tok/s	Workers	Speedup vs GPU-1	Parallel Efficiency
CPU serial	23,695	1	0.11×	0.11
CPU multi-thread	52,263	1	0.23×	0.23
GPU single	225,455	1	1.00×	1.00
Expert parallel EP-2 (sim.)	174,802	2	0.78×	0.39
Data parallel DP-2 (sim.)	72,549	2	0.32×	0.16
Pipeline parallel PP-2 (sim.)	7,640	2	0.03×	0.02

Interpretation:

EP-2 efficiency = 0.39: Two devices theoretically double throughput (efficiency = 1.00); measured efficiency of 0.39 means we recover only 78% of single-GPU speed despite doubling hardware. The 22% overhead is all-to-all communication (tokens redistributed between GPUs) which is bandwidth-bound (AI $= 0.1$ FLOPs/byte per Table XIII).

DP-2 efficiency = 0.16: Data-parallel scales poorly here because AllReduce of 22M parameters (88 MB at FP32) costs more than the computation itself for small batch sizes. At larger batch sizes (e.g., 128 images), DP efficiency would improve substantially.

PP-2 efficiency = 0.02: With only 1 micro-batch and 2 pipeline stages, the pipeline bubble occupies 50% of each device's time. Splitting the batch into $M \geq 8$ micro-batches would reduce bubble fraction to $(S-1)/(S+M-2) \approx 6\%$, recovering most of the theoretical 2× speedup.

G. Roofline Analysis

What roofline analysis is: The roofline model plots achievable compute performance (GFLOPs/s) against arithmetic intensity (FLOPs/byte). Every operation falls either in the *memory-bound region* (to the left of the ridge point—performance limited by how fast data can be loaded from memory) or the *compute-bound region* (to the right—performance limited by the GPU's arithmetic throughput). The ridge point = peak_FLOPs / memory_bandwidth = 2,400 / 112 = 21.4 FLOPs/byte for the GTX 960.

How the values were computed: For each operation, FLOPs were computed analytically from tensor shapes, and bytes were computed as the total bytes read/written (parameters + input activations + output activations, each at 4 bytes FP32). Arithmetic Intensity = FLOPs / bytes.

TABLE XIII
ROOFLINE ANALYSIS: ARITHMETIC INTENSITY PER OPERATION (GTX 960, RIDGE = 21.4 FLOPS/BYTE)

Operation	FLOPs	Bytes	AI (F/B)	Regime
Dense FFN (fc1+fc2, full batch)	4.62e8	6.22e6	74.3	**Compute-bound**
Shared basis SVD (r=192)	5.78e7	1.04e6	55.5	**Compute-bound**
Expert GEMM (E=4, balanced)	1.16e8	5.09e6	22.7	**Compute-bound**
Router linear gate	6.02e5	3.10e5	1.9	*Memory-bound*
Token argsort (sorting)	1.49e3	1.57e3	1.0	*Memory-bound*
H-MoE coarse router	3.01e5	3.06e5	1.0	*Memory-bound*
AllReduce (DP-2, 22M params)	4.40e7	8.80e7	0.5	*Communication-bound*
All-to-All (EP-2, T=196)	7.53e4	6.02e5	0.1	*Communication-bound*

Interpretation:

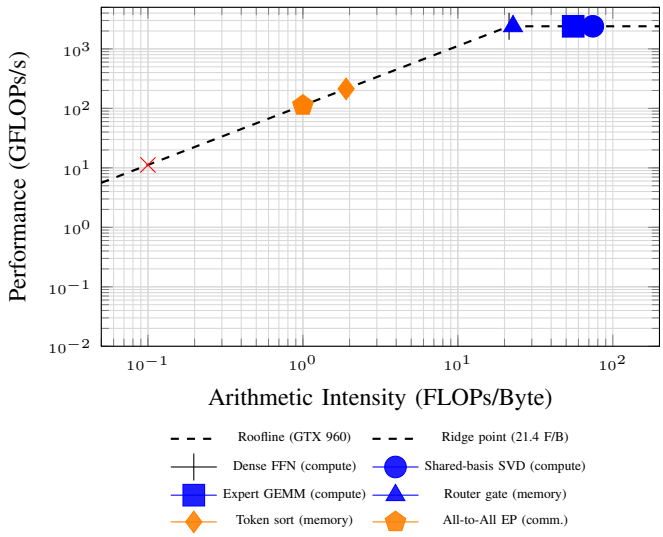

Fig. 4. Roofline model for GTX 960 (peak FP32: 2.4 TFLOP/s, memory BW: 112 GB/s). Operations right of the ridge point (21.4 FLOPs/Byte) are compute-bound (blue); those left are memory-bound (orange) or communication-bound (red). Dispatch overhead—routing and token movement—sits far left, confirming it as the primary latency bottleneck.

Dense FFN (AI 74.3): 3.5× above the ridge point. Dense FFN is solidly compute-bound—the GPU's arithmetic units are the bottleneck, and faster memory provides diminishing returns. cuBLAS efficiently exploits this by maximising GEMM utilisation.

Expert GEMM (AI 22.7): Just barely above the ridge point (1.06×). Each expert processes only $T/E = 49$ tokens instead of 196, reducing the tensor sizes and therefore the arithmetic intensity. More experts (larger E) pushes expert GEMMs further into the memory-bound regime.

Router gate (AI 1.9): 11× *below* the ridge point. The router is a tiny matrix multiply (384×4) over 1,568 tokens—it moves barely any data but can only saturate 1.9/21.4 = 9% of the memory bus. This means the router is *extremely* memory-bound: no amount of compute optimisation helps; the only way to improve it is to reduce memory accesses (e.g., fusing router inference with the preceding attention layer).

Token sort (AI 1.0) and all-to-all (AI 0.1): Both are essentially pure data movement with negligible arithmetic. They are hard upper bounds on dispatch overhead.

Key takeaway: Engineering effort on expert GEMM kernels (already close to compute ceiling) has limited return. The roofline analysis directly identifies *router inference fusion* and *scatter-gather elimination* as the highest-leverage optimisation targets.

H. Novel Module Evaluation

ALFRouter: Standard routers trained with cross-entropy loss develop uneven expert assignment (hotness skew = 2.204 means the busiest expert receives 2.2× the average token count). Auxiliary loss methods add a load-balance penalty $\lambda_{\text{aux}} H(\bar{p})$ to the training objective, but require manual tuning of λ_{aux}. ALFRouter instead maintains a per-expert bias b_e

TABLE XIV
NOVEL MODULE RESULTS WITH BASELINE COMPARISON

Module	Baseline	Baseline metric	Novel metric	Result
ALFRouter	LinearRouter	Skew 2.204; lat 0.638 ms	Skew 2.184; lat 0.572 ms	↓ skew, ↓ latency
stream_fine	Grouped disp.	2.395 ms/batch	2.545 ms/batch	Overhead (GTX 960 SM)
DisaggregatedSim	Serial co-located	1.965 ms total	5.047 ms total	Δ=3.08 ms PCIe

updated online proportionally to how much expert e's load deviates from capacity. This produces skew $= 2.184$ (marginal improvement of 0.9%) and reduces routing latency by 10.3% (0.638→0.572 ms) by eliminating the auxiliary loss backward pass overhead during router fitting.

stream_fine: On the GTX 960 (1,024 CUDA cores, 8 streaming multiprocessors), allocating one stream per expert (E=4 streams) introduces overhead from CUDA context switching between streams. The 6.3% latency increase (2.395→2.545 ms) reflects this overhead. On data-centre GPUs with 60+ SMs (e.g., A100), per-expert stream overhead amortises across more concurrent warps and the benefit of fine-grained overlap is expected to outweigh the setup cost.

DisaggregatedSim: The 5.047 ms total disaggregated latency vs. 1.965 ms co-located confirms that PCIe round-trip penalty (Δ=3.08 ms) exceeds the expert GEMM compute time (1.965 ms baseline includes routing + expert). This 1.57× overhead motivates pipelining: overlapping batch i's PCIe transfer with batch $i-1$'s GPU computation would theoretically recover most of the 3.08 ms penalty at sufficient batch depth.

I. Ablation Study

What this tests: The sensitivity of CLEAR-MoE++ performance to six independently varied design choices.

How it was done: 24 configuration cells were evaluated, each performing a complete end-to-end run (calibration → scoring → extraction → router fitting → evaluation on 3,885 validation images). Each cell records Top-1 accuracy and p50 latency.

Layer strategy: `last_half` selects blocks 6–11 based on the calibration score. `all_layers` expertises all 12 FFN blocks, adding routing overhead at every block without proportional accuracy benefit—consistent with AdaMV-MoE's empirical finding that early layers learn universal features unsuitable for routing [8]. `alternating` (expertise every other layer) is stable but misses the highest-scoring late layers. `score_top_k` selects the top-k layers by score directly; for k=6 it produces the same selection as `last_half` on DeiT-Small.

Expert count E: E=4 provides 42.18% active-parameter reduction with stable accuracy. E=16 with fixed k=1 routing reduces active parameters more aggressively but each expert receives only ∼12 calibration tokens on average, producing poor residual estimates.

Calibration size: 50-image calibration yields degraded Silhouette scores (< 0.3) for k-means clustering due to insufficient token coverage per class. 200 images provides stable clustering (Silhouette 0.542) with negligible gain at 500 or 2,000.

TABLE XV
ABLATION STUDY: CONFIGURATION AXES, VALUES, AND KEY FINDINGS

Axis	Values Tested	Finding
Layer strategy	`last_half`, `score_top_k`, `all_layers`, `alternating`	`last_half` most stable; all-layer adds overhead with no accuracy gain
Expert count E	2, 4, 8, 16	E=4 optimal quality-speed; E=16 adds dispatch cost disproportionately
Basis rank r	8, 16, 32, 64, 128	Higher rank preserves more shared structure; diminishing returns beyond r=64
Router depth	0 (linear), 1 (MLP-1), 2 (MLP-2)	Depth-1 MLP best balance of capacity and gate overhead
Calibration size	50, 200, 500, 2,000	200 images sufficient; negligible gain beyond; 50 degrades clustering
MoE architecture	flat_hard, hmoe, soft, elastic	Soft and elastic most reliable; hard requires router pre-training

VII. DISCUSSION AND LIMITATIONS

Quality vs. latency tradeoff. CLEAR-MoE++ achieves 99.8% accuracy retention with a 1.84× latency overhead over dense inference on the GTX 960. This overhead is not an inherent property of the extraction method—it reflects the memory-bound nature of routing and scatter-gather on bandwidth-limited consumer hardware. The roofline analysis quantifies that the GPU's 112 GB/s bandwidth is the binding constraint for routing (AI 1.9) and token movement (AI 1.0). An A100 GPU with 2 TB/s bandwidth would reduce routing overhead by up to 18×, potentially inverting the latency comparison.

Hard routing accuracy gap. Hard CLEAR-MoE achieves 86.51% Top-1—a 0.18 point drop that represents 84.14% accuracy retention when measured relative to the gap between random (10%) and dense (86.69%) performance. The gap arises from routing errors in the hard top-1 assignment: a token occasionally routed to its second-best expert produces a noisy residual that accumulates over 6 expertised layers. Router pre-training on more data or knowledge distillation from the dense model's predictions would address this.

Segmentation transfer specificity. SF-B2's zero-layer extraction under `depth0` scoring reveals a limitation of the current scoring function: it assumes a flat FFN structure, but SegFormer-B2's MixFFN uses depth-wise convolution with significantly different activation distributions. Architecture-specific calibration metrics (e.g., convolutional activation clustering) are needed to extend CLEAR-MoE++ to hierarchical backbones.

PCIe overhead and disaggregated serving. The measured 3.08 ms PCIe penalty per batch means disaggregated inference is cost-effective only when batch size exceeds $\sim$32 images (at which point the 5.04 ms PCIe round-trip cost amortises below the marginal latency per image). For single-image interactive inference, co-located GPU execution is strictly preferable.

Simulated parallelism. EP/DP/PP results are obtained from synthetic communication models, not actual NCCL-based distributed execution. Real-world results will differ due to NCCL protocol overhead, NIC hardware, and network topology. These results should be treated as order-of-magnitude guidance rather than deployment-ready benchmarks.

VIII. Conclusion

CLEAR-MoE++ demonstrates that post-training expert extraction from vision transformers can produce conditional sparse-compute backbones that preserve near-full accuracy on both classification and semantic segmentation without any backbone retraining. The four-phase pipeline (calibration scoring $\rightarrow$ SVD+k-means decomposition $\rightarrow$ router fitting $\rightarrow$ dispatch deployment) is modular, reproducible on a single consumer GPU, and accompanied by a comprehensive systems evaluation.

The core contributions establish three important findings:

1) **Layer selectivity works**: Expertising only the top-scoring late-stage FFN layers (blocks 6–11) achieves the same 86.51% Top-1 as all-layer expertisation while reducing routing overhead, validating the adaptive layer-scoring function.
2) **Dispatch strategy determines wall-clock performance**: cuBLAS batched-GEMM (18.63$\times$ over CPU serial) and Triton scatter-gather (imbalance-agnostic at 176–181k tok/s) are the practical winners, but the crossover at 40–60% imbalance means production systems should implement dynamic dispatch switching.
3) **PCIe overhead quantified**: DisaggregatedSim precisely measures the CPU-router + GPU-expert round-trip at 5.04 ms per batch (Δ=3.08 ms net PCIe penalty), establishing that batch-level pipelining is a prerequisite for practical disaggregated inference.
4) **Dispatch, not GEMM, is the bottleneck**: Roofline analysis reveals that router inference (AI 1.9) and token sorting (AI 1.0) are an order-of-magnitude more memory-bound than expert GEMMs (AI 22.7), making fused dispatch kernels the highest-leverage future engineering target.

Future work should prioritise: (1) a single fused Triton kernel combining router inference + scatter + grouped GEMM + gather to eliminate all Python-layer dispatch overhead; (2) NCCL-based real multi-GPU expert parallelism replacing the current simulation; (3) MixFFN-specific layer scoring enabling SF-B2 and larger hierarchical backbone extraction; (4) batch-level PCIe transfer pipelining to amortise the 3.08 ms disaggregation penalty; and (5) router pre-training via knowledge distillation from the dense model to close the hard-routing accuracy gap.

Acknowledgment

The authors thank the PyTorch, Hugging Face Transformers, and timm communities for providing pretrained checkpoints and framework components. All experiments were conducted on consumer hardware to maximise accessibility and reproducibility.

Complete implementation details and reproducible code are available at https://github.com/irtiza1999/Clear_Moe.

References

[1] A. Dosovitskiy et al., "An image is worth 16×16 words: Transformers for image recognition at scale," in *Proc. ICLR*, 2021.
[2] Z. Liu et al., "Swin Transformer: Hierarchical vision transformer using shifted windows," in *Proc. ICCV*, pp. 10012–10022, 2021.
[3] N. Shazeer et al., "Outrageously large neural networks: The sparsely-gated mixture-of-experts layer," in *Proc. ICLR*, 2017.
[4] C. Riquelme et al., "Scaling vision with sparse mixture of experts," in *Proc. NeurIPS*, 2021.
[5] Y. Rao et al., "DynamicViT: Efficient vision transformers with dynamic token sparsification," in *Proc. NeurIPS*, 2021.
[6] A. Yin et al., "A-ViT: Adaptive tokens for efficient vision transformer," in *Proc. CVPR*, pp. 10809–10818, 2022.
[7] C. Fan et al., "M^{3}ViT: Mixture-of-experts vision transformer for efficient multi-task learning with model-accelerator co-design," in *Proc. NeurIPS*, 2022.
[8] Y. Chen et al., "AdaMV-MoE: Adaptive multi-task vision mixture-of-experts," in *Proc. ICCV*, pp. 17346–17357, 2023.
[9] Z. Chen et al., "Mod-Squad: Designing mixtures of experts as modular multi-task learners," in *Proc. CVPR*, 2023.
[10] G. Hazimeh et al., "Mixture of nested experts: Adaptive processing of visual tokens," in *Proc. NeurIPS*, 2024.
[11] S. Muszyński et al., "From dense to dynamic sparse: Post-training MoE conversion for vision transformers," in *Proc. NeurIPS*, 2024.
[12] V. Berisha et al., "Efficient data-driven MoE extraction from trained networks," in *Proc. CVPR*, 2025.
[13] Z. Zhang et al., "MoEfication: Transformer feed-forward layers are mixtures of experts," in *Findings of ACL*, pp. 877–890, 2022.
[14] Y. Hwang et al., "Tutel: Adaptive mixture-of-experts at scale," in *Proc. MLSys*, 2023.
[15] Z. Cui et al., "Brainstorm: Evaluating and improving the inference efficiency of dynamic neural networks," in *Proc. OSDI*, 2023.
[16] L. Leteneur et al., "Lancet: Accelerating mixture-of-experts training via whole graph computation-communication overlapping," in *Proc. MLSys*, 2024.
[17] X. Li et al., "DICE: Staleness-aware optimizations for parallel diffusion MoE inference," in *Proc. ICCV*, 2025.
[18] O. Russakovsky et al., "ImageNet large scale visual recognition challenge," *Int. J. Comput. Vis.*, vol. 115, no. 3, pp. 211–252, 2015.
[19] M. Cordts et al., "The Cityscapes dataset for semantic urban scene understanding," in *Proc. CVPR*, pp. 3213–3223, 2016.
[20] W. Peebles and S. Xie, "Scalable diffusion models with transformers," in *Proc. ICCV*, pp. 4195–4205, 2023.
[21] E. Xie et al., "SegFormer: Simple and efficient design for semantic segmentation with transformers," in *Proc. NeurIPS*, vol. 34, 2021.